UNIT II – Search Techniques

AI & ML for Mechanical Engineering (23A03701)

1. Problem-Solving Agents

A ***problem-solving agent*** is a goal-based agent that decides what to do by finding a ***sequence of actions*** that leads to a desired goal state.

➤ Steps followed by the agent

- ***Goal formulation*** – decide which states are "goal states" (e.g. reach the workshop).
- ***Problem formulation*** – decide which actions & states to consider.
- ***Search*** – look for the sequence of actions that reaches the goal.
- ***Execution*** – carry out the actions of the solution one by one.

Cycle: ***Formulate*** → ***Search*** → ***Execute*** (the agent ignores percepts while executing – "open-loop" system).

➤ Well-defined problem has 5 parts

- ***Initial state*** – where the agent starts, e.g. In(Arad).
- ***Actions*** – set of possible actions, e.g. Go(Sibiu).
- ***Transition model*** – RESULT(s,a) = new state.
- ***Goal test*** – is the current state a goal?
- ***Path cost*** – numeric cost of a path = sum of step costs c(s,a,s').

Solution = path from initial state to goal state. ***Optimal solution*** = the one with lowest path cost.

> ***M.E. example:*** Robot arm in a welding shop must move from home position to a weld point avoiding fixtures. State = (joint angles), action = rotate a joint by a small step, goal = tool-tip at weld point, path cost = total joint rotation / time.

2. Searching for Solutions

The possible action sequences form a ***search tree***: root = initial state, branches = actions, nodes = states.

- ***Expanding*** a node = applying all legal actions, generating child nodes.
- ***Frontier (open list)*** = set of generated nodes not yet expanded.
- ***Explored set (closed list)*** = nodes already expanded – avoids repeated states / loops.
- ***Search strategy*** = rule for choosing which frontier node to expand next.

A ***node*** stores: STATE, PARENT, ACTION, PATH-COST g(n).

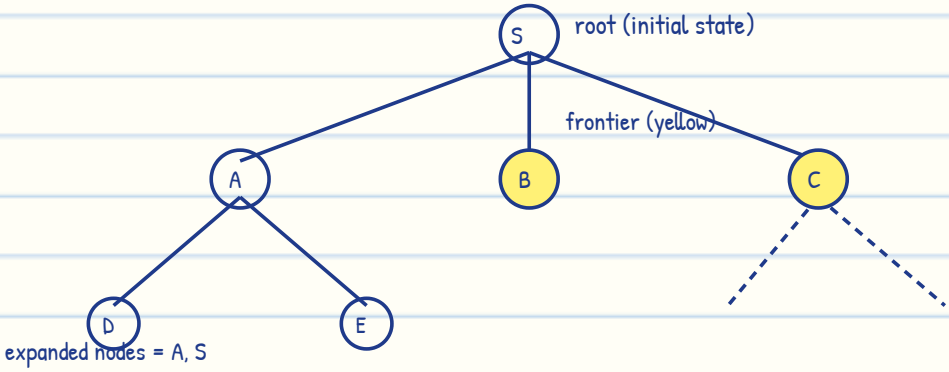


Fig 2.1 – A search tree: S expanded, A expanded, B & C in frontier

➤ Tree search vs Graph search

Tree search	Graph search
No explored set; may revisit same state many times	Keeps explored set; each state expanded once
Can loop forever on cycles	Avoids loops & redundant paths
Less memory	More memory

➤ Measuring problem-solving performance

- ***Completeness*** – is a solution guaranteed if one exists?
- ***Optimality*** – does it find the least-cost solution?
- ***Time complexity*** – number of nodes generated.
- ***Space complexity*** – max nodes stored in memory.

Complexity is expressed with: ***b*** = branching factor (max children), ***d*** = depth of shallowest goal, ***m*** = max depth of state space, ***l*** = depth limit.

3. Uninformed (Uniform / Blind) Search Strategies

These strategies have no extra information about states beyond the problem definition – they only generate successors and test for goal. Also called ***blind search***.

Types: BFS, Uniform-cost search, DFS, Depth-limited, Iterative deepening, Bidirectional.

➤ 3.1 Breadth-First Search (BFS)

Expand the ***shallowest*** unexpanded node first. Frontier is a ***FIFO queue***.

➤ Algorithm

1. Put the start node in a FIFO queue; explored = { }.
2. If queue empty → FAILURE.
3. Remove first node n. If n is goal → return solution.
4. Expand n; add children not in explored/frontier to the END of queue.
5. Go to step 2.

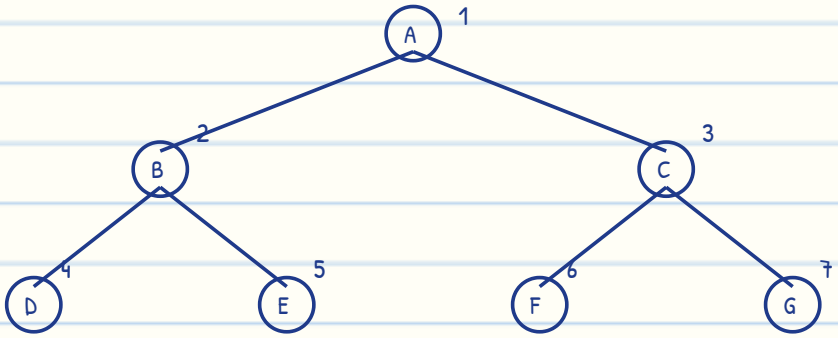


Fig 3.1 – BFS order: A B C D E F G (level by level)

- ***Complete:*** Yes (if b is finite).
- ***Optimal:*** Yes, only when all step costs are equal.
- ***Time:*** $O(b^d)$ ***Space:*** $O(b^d)$ – keeps every node in memory (big drawback).

Example: b = 10, d = 6 → 10^6 nodes; at 1 KB/node ≈ 1 GB. Memory runs out before time does!

Uniform-cost search (UCS): BFS variant that expands the node with the ***lowest path cost g(n)*** (priority queue). Optimal for any positive step cost. Complexity $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$, C^* = optimal cost.

➤ 3.2 Depth-First Search (DFS)

Expand the ***deepest*** unexpanded node first. Frontier is a ***LIFO stack*** (or recursion). Backtracks when a dead-end is reached.

➤ Algorithm

1. Push start node on stack.
2. Pop top node n. If goal → stop.
3. Push its children (left to right) on the stack.
4. Repeat; if stack empty → FAILURE.

For the tree of Fig 3.1, DFS order: **A** → **B** → **D** → **E** → **C** → **F** → **G**.

- ***Complete:*** No – fails in infinite-depth spaces or loops (tree search). Complete in finite graph with explored set.
- ***Optimal:*** No – may return a deeper, costlier solution first.
- ***Time:*** $O(b^m)$ – can be terrible if $m \gg d$.
- ***Space:*** $O(bm)$ – linear, the big advantage.

3.3 Depth-Limited Search (DLS)

DFS with a predetermined ***depth limit l***. Nodes at depth l are treated as if they have no successors → solves the infinite-path problem.

- Returns ***failure*** (no solution) or ***cutoff*** (no solution within limit l).
- ***Complete:*** No if $l < d$ (goal beyond limit). ***Optimal:*** No if $l > d$.
- ***Time:*** $O(b^l)$ ***Space:*** $O(bl)$.
- Limit can come from domain knowledge – e.g. a map with 20 cities → any solution has length ≤ 19, so l = 19 (diameter of the graph gives a better limit).

```
function DLS(node, goal, limit):
  if node == goal: return SOLUTION
  if limit == 0:   return CUTOFF
  for each child of node:
      r = DLS(child, goal, limit-1)
      if r == SOLUTION: return r
  return FAILURE / CUTOFF
```

➤ 3.4 Iterative Deepening DFS (IDDFS)

Repeatedly run DLS with l = 0, 1, 2, 3 … until goal is found. Combines ***BFS completeness/optimality*** with ***DFS memory***.

- Complete: Yes. Optimal: Yes (unit step cost). Time $O(b^d)$. Space O(bd).
- Repeated work is small: most nodes are at the bottom level. Nodes generated = $(d)b + (d-1)b^2 + \dots + b^d$.
- Preferred uninformed method when search space is large and depth of solution unknown.

➤ 3.5 Bidirectional Search

Run ***two searches simultaneously***: one ***forward*** from the initial state, one ***backward*** from the goal; stop when the two frontiers ***meet***.

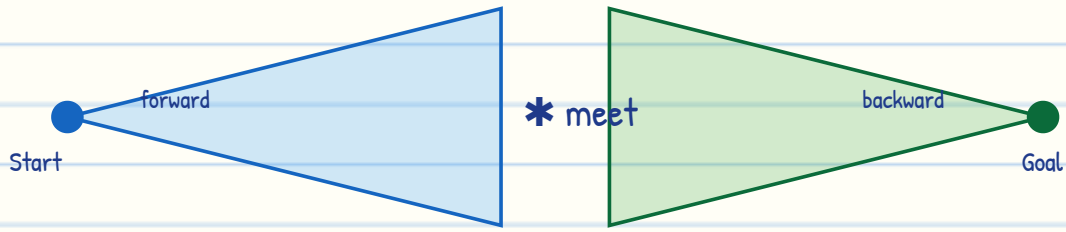


Fig 3.2 – Two small cones ($b^{d/2}$ each) instead of one big cone (b^d)

- ***Time:*** $O(b^{d/2})$ ***Space:*** $O(b^{d/2})$. Example: b=10, d=6 → 2×10^3 = 2000 nodes vs 10^6.
- ***Complete:*** Yes (BFS both sides). ***Optimal:*** Yes (unit costs, BFS both sides).
- ***Requirements / drawbacks:*** goal state must be explicitly known and unique; predecessors must be computable (reversible actions); need an efficient check that frontiers intersect; high memory.

➤ 3.6 Comparing Uninformed Strategies

Criterion	BFS	UCS	DFS	DLS	IDDFS	Bidirectional
Complete?	Yes	Yes	No	No	Yes	Yes
Time	b^d	$b^{1+C^*/\varepsilon}$	b^m	b^l	b^d	$b^{d/2}$
Space	b^d	$b^{1+C^*/\varepsilon}$	bm	bl	bd	$b^{d/2}$
Optimal?	Yes*	Yes	No	No	Yes*	Yes*

** when all step costs are equal. b = branching factor, d = goal depth, m = max depth, l = limit.*

4. Heuristic (Informed) Search Strategies

Informed search uses ***problem-specific knowledge*** beyond the definition, through an ***evaluation function f(n)***, to find solutions more efficiently than blind search.

➤ Heuristic function h(n)

h(n) = ***estimated cost of the cheapest path from node n to a goal***. h(goal) = 0.

- ***Admissible:*** $h(n) \leq h^*(n)$ (never overestimates the true cost).
- ***Consistent (monotonic):*** $h(n) \leq c(n,a,n') + h(n')$.
- Examples: straight-line distance (SLD) in route finding; Manhattan distance in 8-puzzle; number of misplaced tiles.

Notation: ***g(n)*** = cost from start to n, ***h(n)*** = estimated cost n to goal, ***f(n)*** = evaluation function.

➤ 4.1 Greedy Best-First Search

Expands the node that ***appears closest to the goal***: f(n) = h(n). Frontier = priority queue ordered by h.

- ***Complete:*** No (can get stuck in loops, tree version). ***Optimal:*** No.
- ***Time & Space:*** $O(b^m)$ worst case; a good heuristic gives big improvement.
- Advantage: fast, simple. Disadvantage: ignores cost already spent g(n), so can follow a "cheap-looking" but long path.

Example (route): S → A (cost 1), S → B (cost 4). h(A)=7, h(B)=2, h(G)=0. B → G cost 3; A → G cost 9. Greedy picks B (h smaller) then G: path S-B-G, cost 7. Here also optimal, but in general not guaranteed.

➤ 4.2 A* Search

Most popular informed search. Combines cost so far and estimated cost to go:

$$f(n) = g(n) + h(n)$$

➤ Algorithm

1. OPEN = {start}, CLOSED = { }; f(start) = h(start).
2. If OPEN empty → FAILURE.
3. Pick node n in OPEN with ***lowest f(n)***; move it to CLOSED.
4. If n is goal → return path (follow parents).
5. For each successor m: compute g(m) = g(n)+c(n,m), f(m) = g(m)+h(m). If m is new → add to OPEN; if m already present with higher g → update parent & cost.
6. Go to step 2.

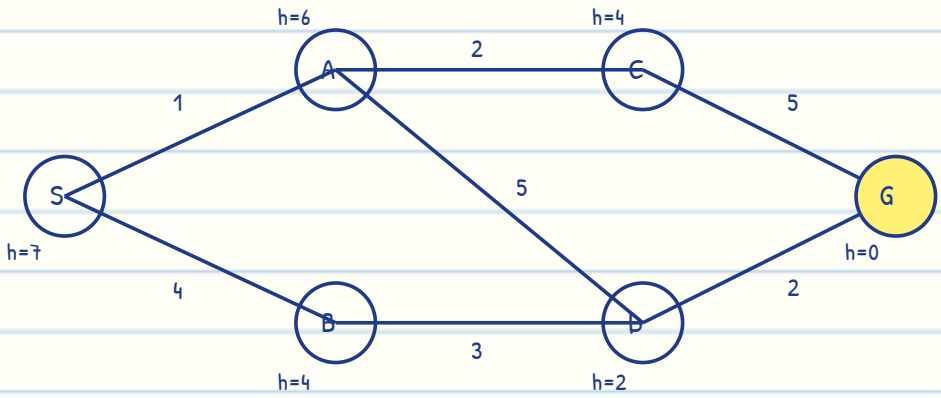


Fig 4.1 – Edge numbers = step cost, red = h(n)

➤ Trace

Step	Expand	OPEN (node : f = g+h)
1	S	A:1+6=7, B:4+4=8
2	A (f=7)	B:8, C:3+4=7, D:6+2=8 (via A)
3	C (f=7)	B:8, D:8, G:8+0=8
4	B (f=8)	D: via B = 4+3+2 = 9 (worse, keep 6+2=8), G:8
5	D (f=8)	G: 6+2=8 → better path via D: g=8
6	G	goal reached, cost = 8

Optimal path: **S** → **A** → **D** → **G** (1+5+2 = 8). Same cost as S-A-C-G (1+2+5=8).

- ***Complete:*** Yes. ***Optimal:*** Yes if h is admissible (tree search) / consistent (graph search).
- ***Time:*** exponential in (h error × solution length). ***Space:*** $O(b^d)$ – keeps all nodes; main drawback.
- A* is ***optimally efficient*** – no other optimal algorithm using same h expands fewer nodes.

Greedy BFS	A*
f = h	f = g + h
Not optimal, not complete	Optimal & complete (admissible h)
Faster, less memory in practice	Slower, heavy memory

4.3 AO* Search (AND-OR graphs)

Used for problems that can be ***decomposed into sub-problems***; solved by an ***AND-OR graph***.

- ***OR node:*** solved if any one child is solved (alternatives).
- ***AND node (AND-arc):*** solved only if all children are solved.
- Solution is not a path but a ***sub-graph (solution graph)***.

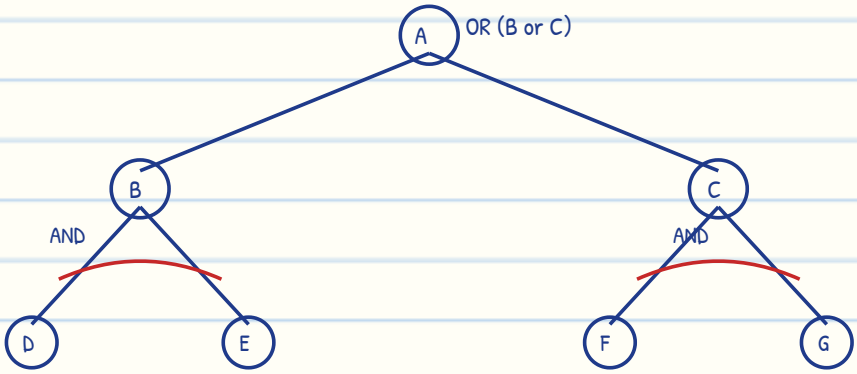


Fig 4.2 – AND-OR graph: A solved via B (needs D AND E) OR via C (needs F AND G)

➤ AO* Algorithm

1. Start with graph containing only the initial node; f = h(start).
2. Trace the ***current best partial solution graph*** (follow marked best arcs) and pick an unexpanded node n on it.
3. Expand n; for each successor compute h. Add to graph.
4. Cost of a node = min over arcs of: ***cost(arc) = Σ (c + h(child)) over AND-children*** (or single child for OR arc). Mark the best arc.
5. ***Back-propagate*** revised costs up to the start node (update all ancestors).
6. Mark node SOLVED when all children on its best arc are solved. Repeat until start is SOLVED or cost > FUTILITY.

- A* works on OR graphs only; AO* handles AND arcs (problem reduction, theorem proving, assembly-sequence planning).
- Optimal if h is admissible; avoids revisiting by cost revision.
- M.E. use: ***assembly planning*** – "assemble gearbox" = (assemble shaft sub-assembly AND assemble housing) OR (buy ready sub-unit).

> Cost eg: arc A → (B AND C) with h(B)=3, h(C)=4, each link cost 1: cost = (1+3)+(1+4) = 9. Arc A → D (OR) with h(D)=10, link 1: cost = 11. AO* selects the AND arc (9).

5. Memory-Bounded Heuristic Search

A* runs out of memory before time. Solutions:

➤ 5.1 IDA* (Iterative-Deepening A*)

- Like IDDFS but cutoff is on ***f-cost*** (g+h) instead of depth.
- Each iteration: DFS, prune nodes with f > current threshold. New threshold = smallest f that exceeded the old one.
- Complete & optimal (admissible h); memory O(bd). Drawback: repeated re-expansion when f-values are all different.

➤ 5.2 RBFS (Recursive Best-First Search)

- Recursive, linear space; keeps track of f-value of best ***alternative*** path available from any ancestor (f_limit).
- If current node's f exceeds f_limit → unwinds, replacing the node's f with best f of its children (remembers "how good this branch was").
- Optimal if h admissible; space O(bd); may re-generate nodes many times.

➤ 5.3 SMA* (Simplified Memory-Bounded A*)

- Works like A* until memory is ***full***; then drops the ***worst leaf*** (highest f) and backs its f-value up to the parent.
- Complete if solution depth fits in memory; optimal if reachable; uses all available memory.

6. Local Search Algorithms & Optimization Problems

In many problems the ***path is irrelevant***; only the final state matters (e.g. n-queens, circuit layout, job-shop scheduling, optimal design of a beam). ***Local search*** keeps a ***single current state*** and moves to neighbours.

- Advantages: very little memory (constant); can find good solutions in huge/infinite spaces.
- Useful for ***optimization problems***: find the state that maximizes/minimizes an ***objective function*** (e.g. minimize weight, maximize efficiency).

➤ State-space landscape

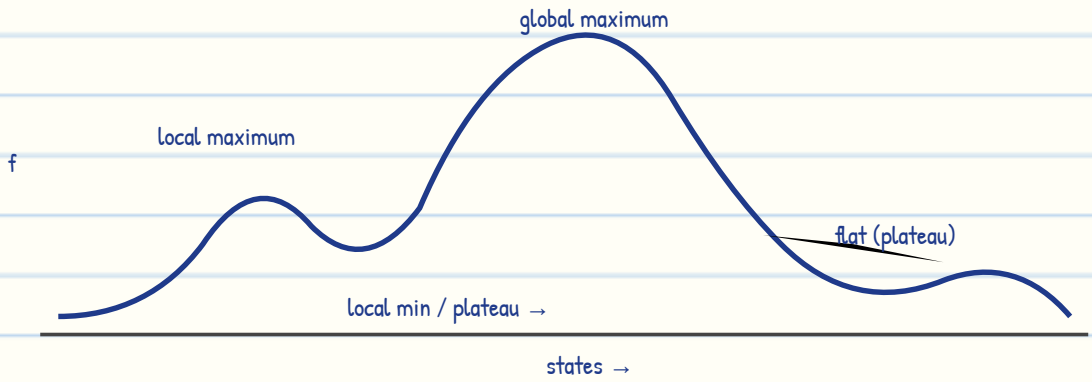


Fig 6.1 – Landscape: shoulder, local max, global max, plateau

➤ 6.1 Hill-Climbing Search

Loop that continually moves in the direction of ***increasing value*** (uphill); terminates when no neighbour has a higher value. Called ***greedy local search*** – no look-ahead beyond immediate neighbours.

```
current = initial state
loop:
  neighbour = highest-valued successor of current
  if value(neighbour) <= value(current): return current
  current = neighbour
```

➤ Problems with hill climbing

- ***Local maxima:*** peak higher than neighbours but lower than global max – search stops.
- ***Ridges:*** sequence of local maxima hard to navigate (needs diagonal moves).
- ***Plateaus / shoulders:*** flat area; no uphill direction – may wander.

➤ Variants

- ***Steepest-ascent:*** checks all neighbours, chooses the best.
- ***Stochastic:*** chooses randomly among uphill moves (probability ∝ steepness).
- ***First-choice:*** generates random successors until a better one appears – good when many neighbours.
- ***Random-restart:*** repeat hill climbing from random initial states; keeps best result – "complete" with probability → 1.

- ***Sideways moves:*** allow limited moves on plateaus.

> ***Eg. 8-queens:*** h = number of attacking pairs. Steepest-ascent solves only ≈14% of random instances, stuck in 86%; with sideways moves ≈94% solved.

➤ 6.2 Simulated Annealing

Inspired by ***annealing in metallurgy*** – heating metal then cooling slowly so atoms settle into a low-energy crystal (strong, defect-free). A very ***mechanical-engineering*** idea!

- Combines hill climbing with ***random walk*** to escape local maxima.
- Picks a ***random*** move. If it improves ($\Delta E > 0$) → always accept. If it worsens → accept with probability $e^{\Delta E/T}$.
- ***Temperature T*** is high at first (many bad moves accepted) and is lowered slowly by a ***cooling schedule***; as $T \to 0$ it behaves like hill climbing.

```
current = initial; T = schedule(0)
for t = 1,2,3...:
   T = schedule(t);  if T == 0: return current
   next = random successor of current
   ΔE = value(next) - value(current)
   if ΔE > 0: current = next
   else: current = next with probability e^(ΔE/T)
```

- If T is lowered slowly enough → finds the ***global optimum*** with probability → 1.
- Applications: VLSI layout, factory/job-shop scheduling, truss & structural optimization, travelling-salesman, tool-path planning.

Hill climbing	*Simulated annealing*
Only uphill moves	Accepts downhill moves with probability
Stuck at local max	Can escape local max
Fast, simple	Slower; needs good cooling schedule

➤ 6.3 Local Beam Search

Keeps track of ***k states*** (the beam width) instead of one.

1. Begin with k randomly generated states.

2. Generate ***all successors*** of all k states.
3. If any is a goal → stop. Else select the ***k best*** successors from the combined list and repeat.

- Differs from k parallel hill-climbs: information is ***shared*** – useful states "recruit" others ("come over here, the grass is greener").
- Problem: states can become concentrated in a small region (lack of diversity).
- ***Stochastic beam search:*** choose k successors randomly with probability proportional to value (like natural selection → leads to ***genetic algorithms***).

Algorithm	Memory	Escapes local max?	Typical use
Hill climbing	O(1)	No	Quick improvement
Simulated annealing	O(1)	Yes (probabilistic)	Layout, scheduling
Local beam	O(k)	Partly	Parallel exploration

7. Quick Revision – Important Questions & Answers

➤ Q1. Differentiate informed and uninformed search.

Uninformed: no domain knowledge, uses only problem definition (BFS, DFS). Informed: uses heuristic h(n) to guide search (Greedy, A*); usually faster & fewer nodes.

➤ Q2. Why is DFS memory efficient but BFS is not?

DFS stores only a single path with siblings → O(bm). BFS stores whole frontier → $O(b^d)$.

➤ Q3. When is A* optimal?

When h(n) is admissible (tree search) or consistent (graph search).

➤ Q4. What is an admissible heuristic? Give an example.

$h(n) \leq$ true cost to goal. e.g. straight-line distance never exceeds the road distance.

➤ Q5. State drawbacks of hill climbing and remedies.

Local maxima, ridges, plateaus. Remedies: random restart, sideways moves, stochastic variants, simulated annealing.

➤ Q6. Why does IDDFS not waste much time repeating?

In a tree with branching factor b most nodes lie at the deepest level, so upper levels re-generated are a small fraction (about 11% overhead for b=10).

➤ Q7. Difference between A* and AO*.

A* searches OR graphs for a path; AO* searches AND-OR graphs for a solution sub-graph where AND arcs require all children solved.

➤ Q8. What is the role of temperature in simulated annealing?

Controls probability $e^{\Delta E/T}$ of accepting bad moves: high T → exploration; low T → exploitation (hill climbing).

➤ Q9. Compare greedy best-first and A*.

Greedy: f=h; fast but incomplete/non-optimal. A*: f=g+h; complete and optimal with admissible h but memory heavy.

➤ Q10. Name memory-bounded heuristic searches.

IDA*, RBFS, SMA*.

➤ Formula Sheet

- BFS: time & space b^d | DFS: time b^m, space bm | DLS: b^l, bl
- IDDFS: b^d, bd | Bidirectional: $b^{d/2}$
- Greedy: f = h | A*: f = g + h | UCS: f = g
- Simulated annealing acceptance probability = $e^{\Delta E/T}$